

Object Oriented Programming in C++

OOP in C++ was introduced to solve this problem by organizing code into classes and objects, making programs easier to understand, reuse, and maintain.

Key Features of OOP in C++:

- (i) Structures code into logical units (classes and objects)
- (ii) Keeps related data and methods together (encapsulation)
- (iii) Makes code modular, reusable and scalable
- (iv) Prevents unauthorized access to data
- (v) Follows the DRY (Don't Repeat Yourself) principle

1. Class

A class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. Using classes, you can create multiple objects with the same behavior instead of writing their code multiple times. In general, class declarations in C++ can include these components.

- (i) **Access Specifiers:** A class can have members defined as public, private, or protected to control accessibility.
- (ii) **Class Name:** The class name should follow naming conventions, usually starting with a capital letter.
- (iii) **Body:** The class body is enclosed with braces {} and defines data members and member functions.

2. Object

An Object is a basic unit of Object-Oriented Programming that represents real-life entities. A typical C++ program creates many objects, which interact with each other by invoking methods. The objects are what perform your code, they are the part of your code visible to the user. An object mainly consists of:

- (i) **State:** It is represented by the data members (attributes) of an object. It also reflects the properties of an object.
- (ii) **Member Function:** A member function is a collection of statements that perform some specific task and may return the result to the caller.
- (iii) **Behavior:** It is represented by the member functions of an object. It also reflects the response of an object to other objects.
- (iv) **Identity:** It is a unique name or reference given to an object that enables it to interact with other objects.

```
#include <iostream>
#include <string>
using namespace std;

class Employee {

    // Instance variables
private:
    string name;
    float salary;

public:
    // Constructor
    Employee(string name, float salary) {
        this->name = name;
        this->salary = salary;
    }

    // getters method
    string getName() { return name; }
    float getSalary() { return salary; }

    // setters method
    void setName(string name) { this->name = name; }
    void setSalary(float salary) { this->salary = salary; }
```

```
// Instance method
void displayDetails() {
    cout << "Employee: " << name << endl;
    cout << "Salary: " << salary << endl;
}
};

int main() {
    Employee emp("Geek", 10000.0f);
    emp.displayDetails();
    return 0;
}
```

Output

Employee: Geek

Salary: 10000

3. Abstraction

Abstraction in C++ is the process of hiding the implementation details and only showing the essential details or features to the user. It allows to focus on what an object does rather than how it does it.

In C++ abstraction is achieved using abstract classes (classes that have at least one pure virtual function).

```
#include <iostream>
```

```
using namespace std;
```

```
// Abstract class Vehicle
```

```
class Vehicle {
```

```
public:

    // Abstract methods
    virtual void accelerate() = 0;

    // pure virtual function
    virtual void brake() = 0;

    void startEngine() {
        cout << "Engine started!" << endl;
    }
};

class Car : public Vehicle {
public:

    // Implement abstract methods
    void accelerate() override {
        cout << "Car: Pressing gas pedal..." << endl;
    }

    void brake() override {
        cout << "Car: Applying brakes..." << endl;
    }
};
```

```
int main() {  
  
    // Create object using pointer to abstract class  
    Vehicle* myCar = new Car();  
    myCar->startEngine();  
    myCar->accelerate();  
    myCar->brake();  
  
    delete myCar;  
    return 0;  
}
```

Output

Engine started!

Car: Pressing gas pedal...

Car: Applying brakes...

4. Encapsulation

Encapsulation is defined as the process of wrapping data and the methods into a single unit, typically a class. It is like a protective shield that prevents the data from being accessed by the code outside the shield.

- (i) Technically, in encapsulation the variables or the data in a class is hidden from any other class and can be accessed only through any member function of the class in which they are declared.
- (ii) In encapsulation, the data in a class is hidden from other classes, which is similar to what **data-hiding** does.
- (iii) Encapsulation can be achieved by declaring all the variables in a class as private and writing public methods in the class to set and get the values of the variables.

```
#include <iostream>
#include <string>
using namespace std;
class Employee {
    // Private fields
private:
    int id;
    string name
public:
    // Setter methods
    void setId(int id) {
        this->id = id;
    }
    void setName(string name) {
        this->name = name;
    }

    // Getter methods
    int getId() {
        return id;
    }

    string getName() {
        return name;
    }
};
```

```
int main() {  
    Employee emp;  
  
    // Using setters  
    emp.setId(101);  
    emp.setName("Geek");  
  
    // Using getters  
    cout << "Employee ID: " << emp.getId() << endl;  
    cout << "Employee Name: " << emp.getName() << endl;  
  
    return 0;  
}
```

Output

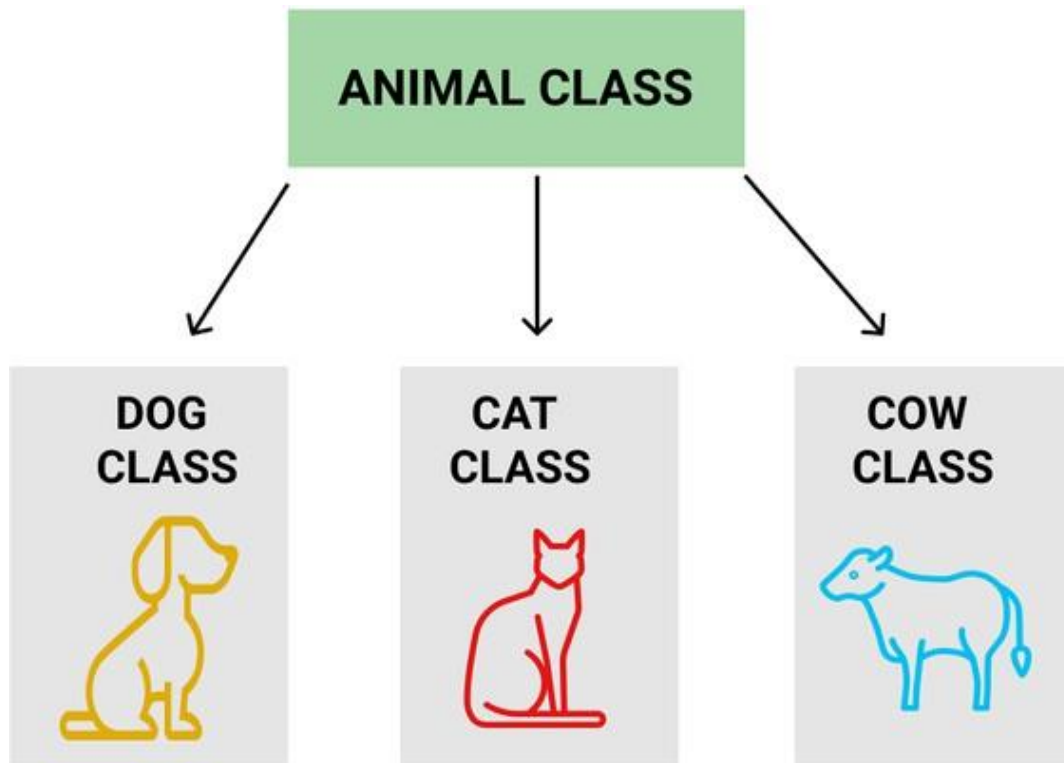
Employee ID: 101

Employee Name: Geek

5. Inheritance

Inheritance is an important pillar of OOP (Object Oriented Programming). It is the mechanism in C++ by which one class is allowed to inherit the features (data members and member functions) of another class. We achieve inheritance by using the : symbol followed by an access specifier (public, private, or protected). Inheritance is also known as an "is-a" relationship.

Example: Dog, Cat, Cow and be Derived Class of Animal Base Class.



Inheritance

Let us discuss some frequently used important terminologies:

- (i) **Superclass:** The class whose features are inherited is known as superclass (also known as base or parent class).
- (ii) **Subclass:** The class that inherits the other class is known as subclass (also known as derived or extended or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- (iii) **Reusability:** Inheritance supports the concept of "reusability", i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class.

```
#include <iostream>

using namespace std;

// Superclass (Parent)
class Animal {
```

```
public:
    void eat() {
        cout << "Animal is eating..." << endl;
    }

    void sleep() {
        cout << "Animal is sleeping..." << endl;
    }
};
```

```
// Subclass (Child) - Inherits from Animal
```

```
class Dog : public Animal {
public:
    void bark() {
        cout << "Dog is barking!" << endl;
    }
};
```

```
int main() {
    Dog myDog;

    // Inherited methods (from Animal)
    myDog.eat();
    myDog.sleep();

    // Child class method
    myDog.bark();
}
```

```
        return 0;  
    }
```

Output

Animal is eating...

Animal is sleeping...

Dog is barking!

6. Polymorphism

The word polymorphism means having many forms, and it comes from the Greek words poly (many) and morph (forms), this means one entity can take many forms. In C++, polymorphism allows the same method or object to behave differently based on the context, specially on the object's actual runtime class.

Types of Polymorphism

Polymorphism in C++ is mainly of 2 types as mentioned below:

- Function Overloading
- Function Overriding

Function Overloading and Function Overriding

1. Function Overloading : Also known as compile-time polymorphism, this occurs when two or more functions in the same class share the same name but have different parameters lists (different number or types of parameters). The return type of these functions may or may not be same.

2. Function Overriding : Also, known as run-time polymorphism, this occurs when a function in the derived class has the same name, return type, and parameters as a function in the base class. In C++, this is achieved using virtual functions, so that the derived class provides its own implementation of the method.